

LC 1345 — Jump Game IV

- Treat indexes as graph nodes.
- Use BFS because every jump costs 1.
- Possible moves: $i-1$, $i+1$, and same-value indexes.
- `seen[]` prevents revisiting indexes.
- `unordered_map<value, vector<indexes>>` stores same-value positions.
- Process each value-group only once, then `clear()`.
- BFS level-by-level $\rightarrow$ step = minimum jumps.

TC: $O(n)$ | SC: $O(n)$

```

class Solution {
public:
    int minJumps(vector<int>& arr) {

        int n = arr.size();
        // arr[i]==arr[j] bhi allowed he hence unordered map saare index with a value store

        unordered_map<int, vector<int>> mp;
        // for value->indexes

        for(int i = 0; i < n; i++){
            mp[arr[i]].push_back(i);
        }

        int step = 0;
        queue<int> q;
        vector<int> seen(n, 0);
        q.push(0);
        seen[0] = 1;

        while(!q.empty()){

            int size = q.size();
            while(size--){
                int curr = q.front();
                if(curr == n - 1) return step;

                q.pop();

                for(auto &it : mp[arr[curr]]){
                    if(!seen[it]){
                        seen[it] = 1;
                        q.push(it);
                    }
                }

                mp[arr[curr]].clear();
                // ek hi value ke indexes ko baar baar process nhi krna

                if(curr + 1 < n) {
                    if(!seen[curr + 1]){
                        seen[curr + 1] = 1;
                        q.push(curr + 1);
                    }
                }

                if(curr - 1 >= 0) {
                    if(!seen[curr - 1]){
                        seen[curr - 1] = 1;

```

```
        q.push(curr - 1);  
    }  
    }  
    }  
    step++;  
}  
  
return step;  
}  
};
```